

Cosc 39: Theory of Computation

Prishita Dharampal

Credit Statement: Talked to Carson Bates'27, Kiran Jones '27, and Sophie Park'27.

Problem 1. Design a regular expression for each of the following languages. No formal proofs required, but explanation of your construction in a few English sentences would be helpful. Throughout the exercise we assume the alphabet to be $\Sigma := \{0, 1\}$.

1. Set of all strings (over $\Sigma := \{0, 1\}$) containing 000 or 111 as substrings.
2. Set of all strings (over $\Sigma := \{0, 1\}$) containing 000 or 111 as subsequences.
3. Set of all strings (over $\Sigma := \{0, 1\}$) not containing 000 nor 111 as substrings.
4. Set of all strings (over $\Sigma := \{0, 1\}$) not containing 000 nor 111 as subsequences.

Solution.

1. $(\Sigma^*000\Sigma^*) + (\Sigma^*111\Sigma^*)$

The first term is the set of all strings containing 000 as a substring, and the second term is the set of all strings containing 111 as a substring. Note that their union also contains all strings with both 000 and 111 as substrings.

2. $(\Sigma^*0\Sigma^*0\Sigma^*0\Sigma^*) + (\Sigma^*1\Sigma^*1\Sigma^*1\Sigma^*)$

Similar to part (1), the first term contains all strings containing 000 as a subsequence and the second term contains all strings containing 111 as a subsequence. And their union contains all strings with both 000 and 111 as subsequences.

3. $((1 + 11) + \epsilon)((0 + 00) \cdot (1 + 11))^*((0 + 00) + \epsilon)$

The expression $((0 + 00) \cdot (1 + 11))^*$ allows us to ensure that there are only runs of maximum length two for each symbol. And the expressions $((1 + 11) + \epsilon)$ and $((0 + 00) + \epsilon)$ allow the string to begin or end with either type symbol or be empty.

4. $((01 + 0) + (10 + 1) + \epsilon)^2 + 0011 + 1100$

Note that since we don't want subsequences of either symbols of length 3, the longest valid strings are of length 4. The expression $((01 + 0) + (10 + 1) + \epsilon)^2$ allows us to construct all but two valid expressions.

Problem ★.

5. Set of all strings containing 000 and 111 as substrings.
7. Set of all strings except for 000 and 111.
8. Set of all strings with more 01s than 10s

Solution.

5. $\Sigma^*000\Sigma^*111\Sigma^* + \Sigma^*111\Sigma^*000\Sigma^*$

Similar to part (1). The first term is the set of all strings containing 000 followed by 111 as substrings, and the second term is the set of all strings containing 111 followed by 111 as substrings.

Note: Let A, B be regular languages, where B is finite. Note that since regular languages are recursively defined as

$$A = \left\{ \bigcup_i x_i \mid \forall x \in A \right\}$$

then for any $x \in A$ we can define

$$A = x + A', \text{ where } A' = \left\{ \bigcup_i x_i \mid \forall x_i \in A, x_i \neq x \right\}$$

Then to determine the set $A - B$ of expressions x such that $x \in A$ and $x \notin B$, disregarding efficiency, we can define $A - B$ as follows:

```
setminus (A, B):  
  A - B = A  
  for (x in B):  
    if (x in A - B):  
      A - B = A'
```

7. $\Sigma^* - \{000, 111\}$

8. $(0^+1^+)^+$